

Modelado UML aplicado a un sistema universitario

Comisión / curso	Lunes tarde	Grupo N.º	Grupo 5
Fecha	1/09/2026	Docente	HERNANDEZ PABLO EDUARDO

Integrantes: Santiago Terminiello, Nahuel Mioni, Emanuel Paoloni, Matias Llanos y Joaquin Valdettaro

Parte 1 - Análisis del dominio

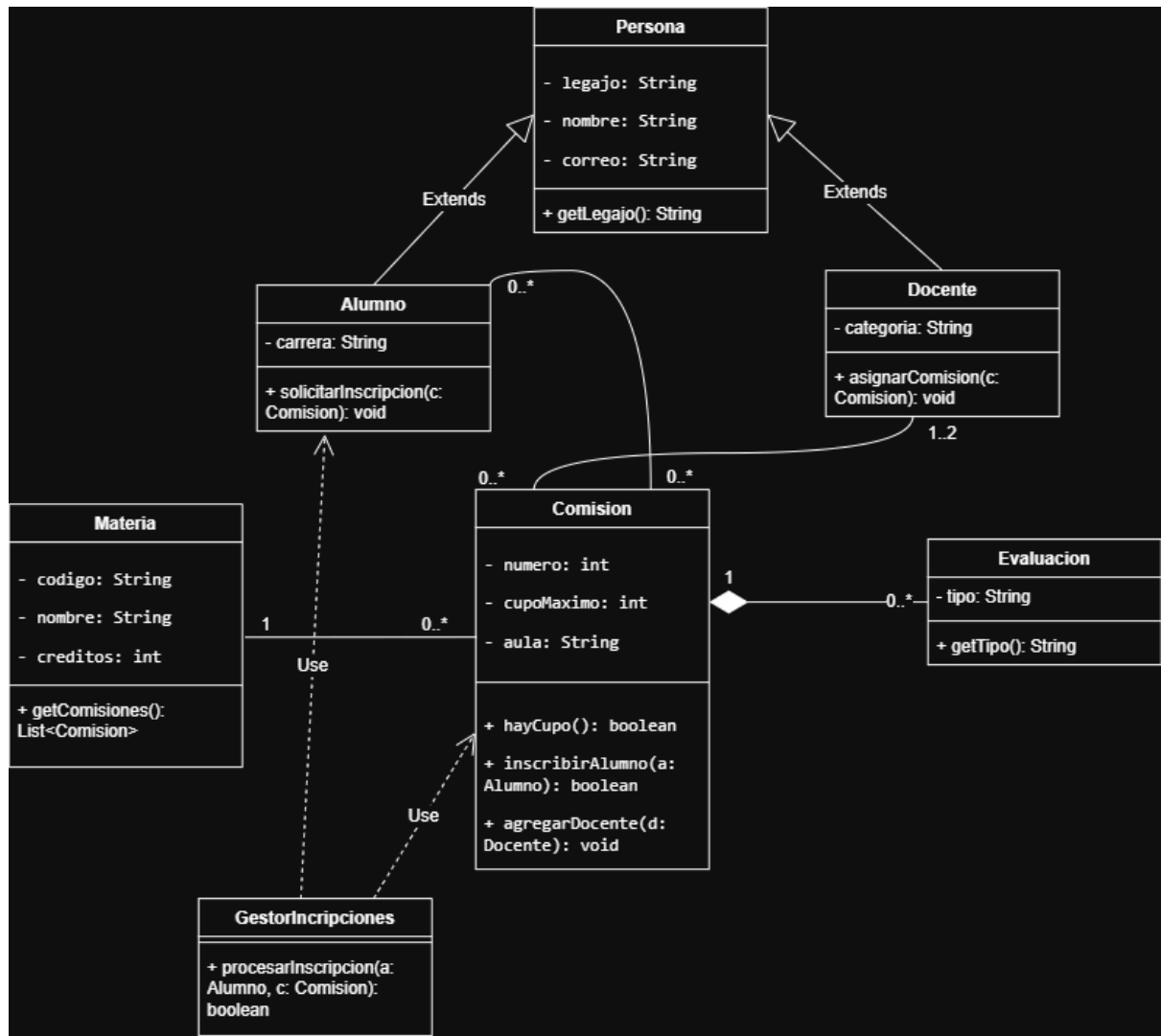
Clases identificadas y su responsabilidad principal:

Clase candidata	Responsabilidad principal
Materia	Encapsular los datos base de la asignatura (código, nombre, créditos) y administrar la colección de comisiones que ofrece.
Comisión	Representar el espacio real de cursada de una materia, vinculando a un grupo de alumnos con sus docentes en un aula específica, y controlar que no se supere el cupo máximo.
Persona	Actuar como superclase para centralizar la estructura de datos comunes (legajo, nombre, correo) y evitar redundancia entre alumnos y docentes.
Alumno	Especializar a Persona incorporando los atributos del perfil estudiantil (carrera) y solicitar la inscripción a las comisiones que desea cursar.
Docente	Especializar a Persona incorporando su categoría docente y mantener el registro de las comisiones que dicta.
Evaluación	Representar cada instancia de evaluación (parcial, recuperatorio o trabajo práctico) definida por una comisión determinada.
Inscripción	Cosificar la relación entre un Alumno y una Comisión, para mantener un registro auditable de la operación (fecha y estado).
GestorInscripciones	Actuar como controlador para ejecutar las reglas de negocio de la inscripción: verificar existencia de la comisión, validar el cupo y controlar la duplicidad.

Conceptos del dominio y roles técnicos. Representan conceptos del dominio Materia, Comisión, Persona, Alumno, Docente, Evaluación e Inscripción: existen en el vocabulario de la universidad con independencia del software. GestorInscripciones cumple un rol técnico o de servicio, ya que no es una entidad del dominio sino el componente que coordina la transacción de inscripción. En el diagrama de secuencia se agregan, con el mismo carácter técnico, PortalWeb (interfaz de usuario) y Repositorio (acceso a los datos persistidos).

Generalización detectada. Persona es la superclase de Alumno y Docente. Corresponde modelarla como herencia porque se verifica la relación "es un/a": todo Alumno es una Persona y todo Docente es una Persona. Ambos comparten legajo, nombre y correo electrónico, de modo que la generalización evita duplicar esos atributos y permite tratarlos de manera uniforme, mientras que carrera y categoría quedan en las subclases porque solo tienen sentido en su especialización.

Parte 2 - Diagrama de clases UML



Justificación 1: relación Comisión - Evaluación

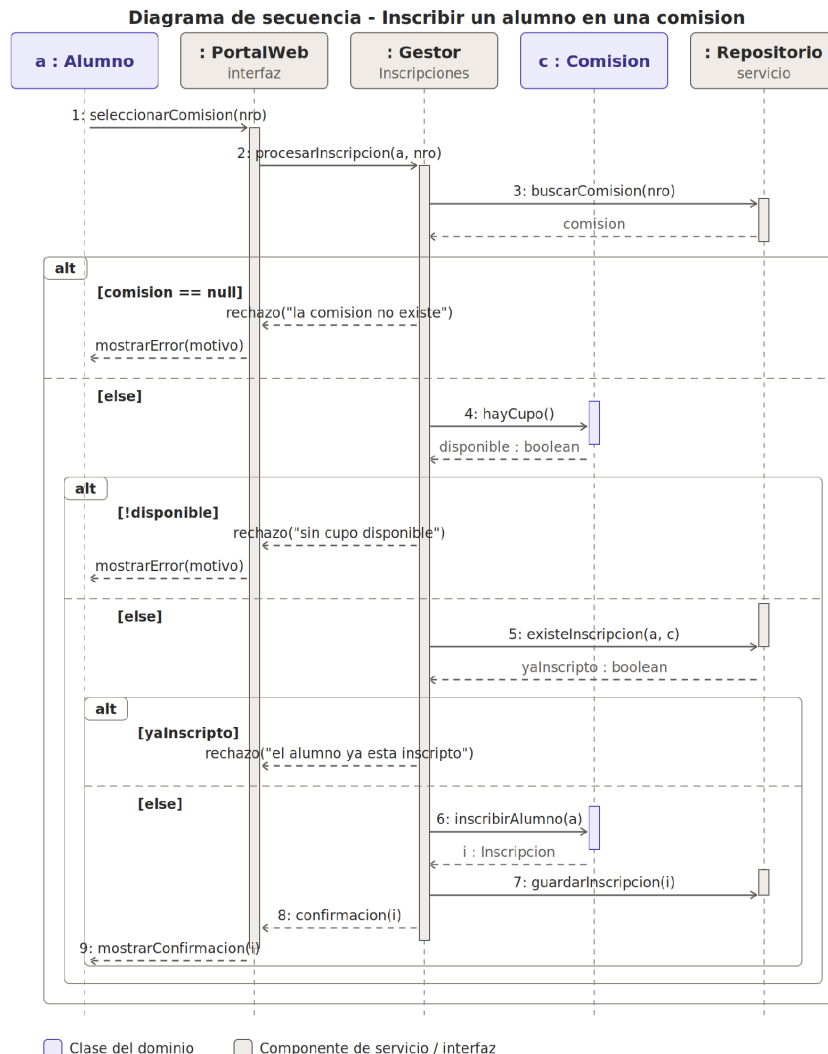
Se modela como composición, con rombo relleno del lado de Comisión y multiplicidad 1 a 0..*. El caso indica que una evaluación no tiene sentido fuera de la comisión para la cual fue creada: la parte no puede existir sin el todo y su ciclo de vida está ligado al del compuesto, de modo que si la comisión se elimina, sus evaluaciones se eliminan con ella. Una agregación sería incorrecta porque implicaría que la evaluación puede existir de forma independiente.

Justificación 2: multiplicidad Docente - Comisión

Se estableció 1..2 en el extremo Docente porque la regla de negocio exige que una comisión nunca se instancie sin profesores y que no supere el límite de dos asignados. Del lado de Comisión se usa 0..*, ya que un docente puede dictar varias comisiones y también puede no tener ninguna asignada en el período.

Parte 3 - Diagrama de secuencia

Caso de uso: inscribir un alumno en una comisión.



Correspondencia con el diagrama de clases. Las líneas de vida `a : Alumno` y `c : Comisión` corresponden a clases del modelo estático. `GestorInscripciones` también figura en el diagrama de clases como controlador. `PortalWeb` y `Repositorio` se identifican explícitamente como componentes de interfaz y de servicio: no son entidades del dominio, sino los responsables de capturar la solicitud del usuario y de acceder a los datos persistidos.

Alternativas modeladas. Los tres fragmentos `alt` anidados respetan el orden real de las verificaciones, de modo que nunca se consulta el cupo de una comisión inexistente. Las alternativas de rechazo son: comisión inexistente, sin cupo disponible y alumno ya inscripto. El flujo principal, en el operando `else` más interno, registra la inscripción, la persiste y devuelve la confirmación al alumno.

Matriz de trazabilidad

Mensaje del diagrama	Objeto receptor	Método o responsabilidad asociada
seleccionarComision(nro)	: PortalWeb	Capturar la solicitud del alumno en la interfaz.
procesarInscripcion(a, nro)	: GestorInscripciones	procesarInscripcion(Alumno, int) : boolean
buscarComision(nro)	: Repositorio	Recuperar la comisión persistida a partir de su número.
hayCupo()	c : Comision	hayCupo() : boolean
existeInscripcion(a, c)	: Repositorio	Controlar que el alumno no esté ya inscripto en esa comisión.
inscribirAlumno(a)	c : Comision	inscribirAlumno(Alumno) : Inscripcion
guardarInscripcion(i)	: Repositorio	Persistir la inscripción registrada.
confirmacion(i) / rechazo(motivo)	: PortalWeb	Informar al alumno el resultado de la operación.

Parte 4 - Del modelo a Java: firmas y wrappers

A. Firmas de métodos

```
public Comision(int numero, int cupoMaximo, String aula, Materia materia)
public boolean hayCupo()
public Inscripcion inscribirAlumno(Alumno alumno)
public int getCupoDisponible()
public boolean procesarInscripcion(Alumno alumno, int nroComision)
```

Firma	Tipo de retorno	Parámetros
Comision(...)	No declara tipo de retorno (constructor)	int numero, int cupoMaximo, String aula, Materia materia
hayCupo()	boolean	Ninguno: opera sobre el estado interno del objeto.
inscribirAlumno(...)	Inscripcion	Alumno alumno
getCupoDisponible()	int	Ninguno: calcula cupoMaximo menos la cantidad de inscriptos.
procesarInscripcion(...)	boolean	Alumno alumno, int nroComision

hayCupo() y getCupoDisponible() son consultas sin parámetros porque el cupo máximo y la colección de inscriptos ya forman parte del estado de la comisión. inscribirAlumno devuelve el objeto Inscripcion en lugar de void para que el gestor pueda persistirlo y devolverlo en la confirmación, tal como se ve en el diagrama de secuencia.

B. Tipos primitivos y wrappers

Dato	Tipo primitivo posible	Wrapper correspondiente	¿Por qué podría convenir el wrapper?
cupoMaximo	int	Integer	Permite el valor null para distinguir un cupo todavía no definido de un cupo igual a 0, y es necesario si el dato viaja en colecciones genéricas como Map<String, Integer>.
cantidadInscriptos	int	Integer	Suele obtenerse de una consulta que puede no devolver filas; el wrapper permite representar ese "sin dato" y usarlo en estructuras como List<Integer>.
promedioFinal	double	Double	Un alumno sin evaluaciones rendidas no tiene promedio: null expresa "sin nota", mientras que 0.0 significaría un promedio real de cero.
regular	boolean	Boolean	Permite un tercer valor lógico: null para la condición aún no determinada, antes del cierre de la cursada.

Parte 5 - Revisión cruzada y entrega

✓	Control final
X	El diagrama de clases contiene clases, atributos, operaciones y relaciones legibles.
X	Todas las asociaciones importantes tienen multiplicidad en ambos extremos.
X	La generalización está orientada desde las subclases hacia la superclase.
X	La relación Comisión-Evaluación está representada y justificada.
X	El diagrama de secuencia tiene líneas de vida, mensajes ordenados y alternativas de error/rechazo.
X	Los nombres de objetos, clases y mensajes son consistentes entre diagramas.
X	Las firmas Java propuestas son compatibles con las responsabilidades del modelo.
X	La tabla de wrappers está completa.
X	Todos los integrantes figuran en la entrega.